

製造指示のアプリケーションモジュール — 機能設計

001_DD-FN — 001_DD と 001_DD-API（後に Screen B の一覧の DD）が使う。要件 REQ-010～REQ-018。

改版履歴

項目	値
ドキュメントID	001_DD-FN
システム名	ProductionManagementAI
サブシステム名	製造指示（Production orders）
作業項目	WI-002
作成者	Claude（ThanhTN 向け）
作成日	2026-09-18
最終更新者	Claude（ThanhTN 向け）
最終更新日	2026-09-22

版	日付	作成者	改版内容
1	2026-09-18	Claude（ThanhTN 向け）	新規作成。001_DD リビジョン 1 のモジュール 3～5（サービス、工場の時計、指示番号の採番）を本書に移す
2	2026-09-18	Claude（ThanhTN 向け）	実装と整合（計画リビジョン 2）：生成された値は再読み込みではなく RETURNING で戻る。読み込んだ <code>xmin</code> を同時実行制御の元の値とする。 <code>AllowedNext()</code> は拡張メソッド
3	2026-09-22	Claude（ThanhTN 向け）	WI-004 REQ-033：UpdateAsync のステップ 7 がエンティティを通して完了日時を記録する。新しいステップも新しい失敗の経路もない

概要とメソッドの索引

項目	値
モジュール名	ProductionOrderService とその補助サービス PlantClock と OrderNumberIssuer （ ProductionManagementAI.Application.ProductionOrders 。 2 つのインターフェースの実装は ProductionManagementAI.Infrastructure.ProductionOrders ）
概要	製造指示のアプリケーション層のユースケース： 入力を検証し、Domain のルールを適用し、リポジトリを通して永続化し、API が HTTP レスポンスに対応付ける型付きの結果を返す。時計と採番は共通なので、Screen B （一覧、納期遅れの表示） が再利用する

No	メソッド名	概要	備考
1	ProductionOrderService.ListProductsAsync	選択用の製品、SKU 順	API-PRD-01
2	ProductionOrderService.GetAsync	1 件の製造指示を ProductionOrderResponse として	API-PO-02
3	ProductionOrderService.CreateAsync	検証、番号の発行、挿入	API-PO-01
4	ProductionOrderService.UpdateAsync	検証、バージョンのチェック、Domain のルール、保存	API-PO-03
5	ProductionOrderMapper.ToResponse	エンティティ → レスポンスの DTO （下の値の加工）	内部、静的
6	IPlantClock.Today / CurrentYear	工場ローカルの日付と年 （ Asia/Tokyo ）	共通
7	IOrderNumberIssuer.NextAsync	カウンターの upsert による年ごとの連番	共通

すべてのサービスのメソッドは CancellationToken を受け取り、Result<T> を返す。これは Ok(T) 、Invalid(errors) 、NotFound 、Conflict 、RuleViolation(code) のバリエーションを持つ判別可能な結果で、コントローラーはそれぞれを 200/201、400、404、409、422 に対応付ける（DEC-023）。想定外の例外は Result にせず、全体の Problem Details のハンドラーに伝える（500、MSG-E013）。

共通のユーティリティの参照

No	名前	概要	備考
1	TimeProvider （.NET）	注入する時刻の源。テストでは FakeTimeProvider	TimeProvider.System として登録
2	IProductionOrderRepository	EF Core のデータアクセス： ListProductsAsync 、 ProductExistsAsync 、 FindAsync(id, tracked) 、 Add 、 SaveChangesAsync 、 BeginTransactionAsync	Infrastructure。列を射影する単一テーブルのクエリ（001_DD「データベースとトランザクションの対応」）
3	ProductionOrderTelemetry	ProductionManagementAI.ProductionOrders という名前の ActivitySource + Meter	001_DD「可観測性」
4	ILogger<ProductionOrderService>	LoggerMessage のソース生成メソッドによる構造化ログ	備考のテキストは決してログに記録しない

リクエストのデータ

本モジュールが（リポジトリと採番を通して）データベースに送るパラメータ。すべてパラメータ化し、SQL に連結するものはない（ `ai/rules/database.md` ）。

No	名前	変数名	型	長さ	使用するメソッド	備考
1	製造指示 ID	<code>id</code>	<code>Guid</code>	—	<code>GetAsync</code> 、 <code>UpdateAsync</code>	主キーでの検索
2	製品 ID	<code>productId</code>	<code>Guid</code>	—	<code>CreateAsync</code> 、 <code>UpdateAsync</code>	<code>ProductExistsAsync</code>
3	指示の年	<code>year</code>	<code>short</code>	—	<code>CreateAsync</code> → <code>NextAsync</code>	カウンターの upsert の <code>@year</code>
4	元の行バージョン	<code>xmin</code>	<code>uint</code>	—	<code>UpdateAsync</code>	EF が <code>WHERE xmin = @original</code> を付ける
5	エンティティの値	<code>ProductionOrder</code>	エンティティ	—	<code>CreateAsync</code> 、 <code>UpdateAsync</code>	EF の変更追跡による INSERT / UPDATE

レスポンス／値の加工

`ProductionOrderResponse`（項目の一覧は 001_DD-API）は `ProductionOrderMapper.ToResponse` が組み立てる：

No	名前	変数名	型	長さ	必須	値の加工	備考
1	ID	<code>Id</code>	<code>Guid</code>	—	yes	<code>entity.Id</code>	
2	指示番号	<code>OrderNumber</code>	<code>string</code>	13	yes	<code>entity.OrderNumber</code> （DB が生成。挿入時に EF が <code>RETURNING</code> で読み戻す）	
3	製品 ID	<code>ProductId</code>	<code>Guid</code>	—	yes	保存どおり	
4	数量	<code>Quantity</code>	<code>int</code>	—	yes	保存どおり	
5	納期	<code>DueDate</code>	<code>DateOnly</code>	—	yes	保存どおり	<code>YYYY-MM-DD</code> に直列化
6	ステータス	<code>Status</code>	<code>string</code>	—	yes	<code>entity.Status.ToString()</code>	
7	遷移可能な次のステータス	<code>AllowedNextStatuses</code>	<code>string[]</code>	—	yes	<code>entity.Status.AllowedNext()</code> （ <code>ProductionOrderStatusExtensions</code> ）、列挙型の順	M-02
8	製品・数量の編集可否	<code>IsProductQuantityEditable</code>	<code>bool</code>	—	yes	<code>entity.Status == Draft</code>	REQ-018
9	備考	<code>Notes</code>	<code>string?</code>	500	yes	保存どおり（空のときは <code>null</code> ）	
10	登録日時	<code>CreatedAt</code>	<code>DateTimeOffset</code>	—	yes	<code>created_at_utc</code>	UTC

No	名前	変数名	型	長さ	必須	値の加工	備考
11	更新日時	UpdatedAt	DateTimeOffset	—	yes	updated_at_utc	UTC
12	バージョン	Version	uint	—	yes	entity.RowVersion (xmin)	DEC-014

ProductResponse = { Id, Sku, Name }。クエリで直接射影する。

メソッドの設計

1. ProductionOrderService.ListProductsAsync

項目	値
説明	製品の選択のために、すべての製品を SKU 順で返す
戻り値の型	Task<IReadOnlyList<ProductResponse>>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	CancellationToken	ct	

戻り値

型	名前	説明
IReadOnlyList<ProductResponse>	products	現在の初期データで 30 行。空のリストも有効（クライアントでは MSG-E014）

処理概要：追跡なしの射影 1 回。ページングなし（DEC-019、DEC-022）。

処理フロー

ステップ	説明	呼び出し
1	<code>SELECT id, sku, name FROM products ORDER BY sku</code>	<code>IProductionOrderRepository.ListProductsAsync</code>
2	リストを返す	—

2. ProductionOrderService.GetAsync

項目	値
説明	編集画面のために 1 件の製造指示を読み込む
戻り値の型	<code>Task<Result<ProductionOrderResponse>></code>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>Guid</code>	<code>id</code>	ルートからの製造指示 ID
2	<code>CancellationToken</code>	<code>ct</code>	

戻り値

型	名前	説明
<code>Result<ProductionOrderResponse></code>	<code>result</code>	<code>Ok</code> または <code>NotFound</code>

処理概要： `xmin` を含む追跡なしの読み込みの後に対応付ける。

処理フロー

ステップ	説明	呼び出し
1	<code>FindAsync(id, tracked: false)</code>	リポジトリ
2	見つからない → <code>NotFound</code>	—
3	<code>Ok(ToResponse(order))</code>	<code>ProductionOrderMapper.ToResponse</code>

3. ProductionOrderService.CreateAsync

項目	値
説明	次の PO-YYYY-NNNNN の番号で Draft の製造指示を登録する (REQ-010)
戻り値の型	Task<Result<ProductionOrderResponse>>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	CreateProductionOrderRequest	request	ProductId?、Quantity?、DueDate?、Notes? (項目の欠落がバインドのエラーではなく「必須」として報告されるよう NULL 許容)
2	CancellationToken	ct	

戻り値

型	名前	説明
Result<ProductionOrderResponse>	result	Ok (コントローラーは 201 + Location を返す) または Invalid(errors)

処理概要：データベースに触れる前にすべての項目エラーを集める。次にデータに依存するチェックを実行する。その後、1 つのトランザクションの中で番号を発行して挿入するので、挿入が失敗しても番号を消費しない (001_DB)。

処理フロー

ステップ	説明	呼び出し
1	スパン ProductionOrder.Create を開始	ProductionOrderTelemetry
2	必須のチェック： ProductId (MSG-E001)、Quantity (MSG-E003)、DueDate (MSG-E004)	—
3	値のチェック： ValidateQuantity (MSG-E003 / MSG-E010)、NormalizeNotes (前後の空白を除去、空 → null) + ValidateNotes (MSG-E006)	ProductionOrder の静的バリデーター (001_DD モジュール 1)
4	データのチェック。エラーのない項目についてだけ実行： ProductExistsAsync (MSG-E002)。DueDate < IPlantClock.Today → MSG-E005	リポジトリ、IPlantClock.Today

ステップ	説明	呼び出し
5	エラーあり → <code>Invalid(errors)</code> 。カウンター <code>created{outcome=validation_failed}</code> 。ログ <code>ProductionOrderValidationFailed</code> （項目名のみ）	—
6	<code>BeginTransactionAsync</code>	リポジトリ
7	<code>seq = NextAsync(IPlantClock.CurrentYear)</code>	<code>IOrderNumberIssuer.NextAsync</code>
8	<code>order = ProductionOrder.Create(productId, quantity, dueDate, notes, year, seq, utcNow)</code>	Domain
9	<code>Add(order)</code> 、 <code>SaveChangesAsync</code> 、コミット	リポジトリ
10	生成された <code>order_number</code> と新しい <code>xmin</code> はすでにエンティティにある：EF Core が <code>SaveChangesAsync</code> の中で <code>RETURNING</code> で読み戻す（追加のクエリなし）	EF Core
11	カウンター <code>created{outcome=success}</code> 。スパンの属性 <code>production_order.id</code> 、 <code>production_order.number</code> 。 <code>Ok(ToResponse(order))</code>	テレメトリー、マッパー

ステップ 7～9 での失敗（例：ある年の 100,000 件目の製造指示でのカウンターの CHECK 違反）→ ロールバック、カウンター `created{outcome=error}`、Error のログ 再スロー → 500 MSG-E013（DEC-013）。

4. ProductionOrderService.UpdateAsync

項目	値
説明	古いデータでの保存を防いで、編集とステータスの変更を保存する（REQ-011、REQ-017、REQ-018、DEC-010）
戻り値の型	<code>Task<Result<ProductionOrderResponse>></code>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>Guid</code>	<code>id</code>	ルートの ID
2	<code>UpdateProductionOrderRequest</code>	<code>request</code>	<code>ProductId?</code> 、 <code>Quantity?</code> 、 <code>DueDate?</code> 、 <code>Status?</code> 、 <code>Notes?</code> 、 <code>Version?</code>
3	<code>CancellationToken</code>	<code>ct</code>	

戻り値

型	名前	説明
<code>Result<ProductionOrderResponse></code>	<code>result</code>	<code>Ok</code> 、 <code>Invalid</code> 、 <code>NotFound</code> 、 <code>Conflict</code> 、 <code>RuleViolation("MSG-E007" \ "MSG-E008")</code>

処理概要：チェックの順序は固定なので、各失敗の結果はちょうど 1 つ：形（400）→ 存在（404）→ バージョン（409）→ Domain のルール（422）→ データに依存するチェック（400）→ 保存時の競合（409）。すべてのチェックに通らない限り何も書き込まない（1 回の `SaveChangesAsync`、DEC-007）。

処理フロー

ステップ	説明	呼び出し
1	スパン <code>ProductionOrder.Update</code> を開始	テレメトリー
2	登録のステップ 2～3 と同じ必須／値のチェック。加えて <code>Status</code> が必須で既知の列挙値であること（未知なら MSG-E007）、 <code>Version</code> が必須（MSG-E009）	Domain のバリデータ
3	エラー → <code>Invalid</code> （ <code>updated{outcome=validation_failed}</code> ）	—
4	<code>FindAsync(id, tracked: true)</code> 。ない → <code>NotFound</code> （ <code>updated{outcome=not_found}</code> ）	リポジトリ
5	<code>request.Version != order.RowVersion</code> → <code>Conflict</code> （ <code>updated{outcome=conflict}</code> ）、ログ <code>ProductionOrderConcurrencyConflict</code> ）	—
6	<code>fromStatus</code> 、 <code>dueDateChanged = request.DueDate != order.DueDate</code> 、 <code>productChanged = request.ProductId != order.ProductId</code> を覚える	—
7	<code>order.Update(productId, quantity, dueDate, notes, status, utcNow)</code> 。 <code>DomainRuleViolation(code)</code> を捕捉 → <code>RuleViolation(code)</code> （ <code>updated{outcome=rule_violation}</code> ）、ログ <code>ProductionOrderRuleViolated</code> ）。例外をスローしたときエンティティは変わらない。 <code>InProgress</code> → <code>Completed</code> では、エンティティは <code>CompletedAtUtc = utcNow</code> も設定する（001_DD モジュール 1 ステップ 5、WI-004 REQ-033）。後のステップが失敗すると何も保存されないで、完了日時は記録されない	Domain（001_DD モジュール 1）
8	<code>productChanged</code> なら <code>ProductExistsAsync</code> （MSG-E002）。 <code>dueDateChanged</code> なら <code>DueDate < IPlantClock.Today</code> → MSG-E005（DEC-009）。エラー → <code>Invalid</code> 。追跡した変更は破棄される（コンテキストはリクエストの範囲で、保存しない）	リポジトリ、 <code>IPlantClock</code>
9	<code>SaveChangesAsync</code> 。EF は <code>WHERE xmin = @original</code> を付ける。元の値はステップ 4 で読み込んだ <code>xmin</code> （ステップ 5 で <code>request.Version</code> と等しいことを確認済み）なので、ステップ 4 と 9 の間の書き込みも捕捉される	リポジトリ
10	<code>DbUpdateConcurrencyException</code> → <code>Conflict</code>	—
11	<code>status != fromStatus</code> なら：カウンター <code>status_transitions{from,to}</code> 、スパンの <code>status.from / status.to</code>	テレメトリー
12	新しい <code>xmin</code> は <code>RETURNING</code> で戻る → <code>Ok(ToResponse(order))</code> 。 <code>updated{outcome=success}</code>	マッパー

5. ProductionOrderMapper.ToResponse

項目	値
説明	エンティティから <code>ProductionOrderResponse</code> への純粋な対応付け
戻り値の型	<code>ProductionOrderResponse</code>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>ProductionOrder</code>	<code>order</code>	読み込んだエンティティ

戻り値

型	名前	説明
<code>ProductionOrderResponse</code>	<code>response</code>	「レスポンス／値の加工」を参照

処理概要：項目ごとのコピーと 2 つの計算した項目。I/O なし。

処理フロー

ステップ	説明	呼び出し
1	保存された項目をコピー	—
2	<code>AllowedNextStatuses</code> と <code>IsProductQuantityEditable</code> を計算	<code>ProductionOrderStatusExtensions.AllowedNext</code>

6. IPlantClock.Today / CurrentYear

項目	値
説明	<code>Plant.TimeZone</code> （既定 <code>Asia/Tokyo</code> 、DEC-011、DEC-017）で設定したタイムゾーンでの工場ローカルの日付と年
戻り値の型	<code>DateOnly</code> / <code>short</code>
作成者／日付	Claude / 2026-09-18

項目	値
最終更新者／日付	—

引数：なし。コンストラクターの依存： `TimeProvider`、`IOptions<PlantOptions>`。

戻り値

型	名前	説明
<code>DateOnly</code>	<code>Today</code>	<code>DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(timeProvider.GetUtcNow(), zone).DateTime)</code>
<code>short</code>	<code>CurrentYear</code>	<code>(short)Today.Year</code>

処理概要： `TimeZoneInfo.FindSystemTimeZoneById(options.TimeZone)` は構築時に一度解決する。.NET の ICU のサポートにより、IANA の ID は Linux と Windows の両方で動く。起動時のオプションの検証が未知の ID を拒否するので、アプリはリクエストごとではなく起動時にすぐ失敗する。

処理フロー

ステップ	説明	呼び出し
1	現在の UTC 時刻を読む	<code>TimeProvider.GetUtcNow</code>
2	工場のタイムゾーンに変換し、日付／年を取る	<code>TimeZoneInfo.ConvertTime</code>

7. IOrderNumberIssuer.NextAsync

項目	値
説明	呼び出し元のトランザクションの中で、001_DB のカウンターの upsert を使ってその年の次の <code>order_seq</code> を返す（DEC-013）
戻り値の型	<code>Task<int></code>
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	<code>short</code>	<code>year</code>	工場の年

No	型	名前	説明
2	CancellationToken	ct	

戻り値

型	名前	説明
int	seq	1～99999

処理概要： Database.SqlQuery<int>(\$"INSERT ... ON CONFLICT (order_year) DO UPDATE SET last_seq = ... + 1 RETURNING last_seq")。EF は補間した SQL をパラメータに変換する。行のロックは呼び出し元がコミットするまで保持され、同じ年の登録を直列化する。

処理フロー

ステップ	説明	呼び出し
1	Database.CurrentTransaction が null なら InvalidOperationException をスロー（挿入のトランザクションの外で番号が発行されるのを防ぐ）	—
2	upsert を実行し、last_seq を返す	001_DB の production_order_number_counters
3	CHECK 違反（last_seq > 99999）は PostgresException 23514 として現れる → 呼び出し元がロールバック → 500	—

未解決の決定

なし。関連する決定： work-items/WI-002/decisions.md の DEC-009、DEC-010、DEC-013、DEC-014、DEC-017、DEC-023。